

可解釋規劃與 TOC 瓶頸分析設計 (Explainable Planning + TOC Bottleneck) — v3.27

本檔性質：跨作業研究 / 演算法 / ERP / 可解釋 AI 四域之方法論文件，描述 erpilot v3.27 之規劃可解釋性 (Explainable Planning) 引擎與 Goldratt (1984) Theory of Constraints (TOC) 瓶頸分析模組之設計、實作與驗證。

■ 前置文件：[MRP_ALGORITHM_DESIGN_ZH.md](#) (v3.25.10) / [MRP_CAPACITY_AWARE_DESIGN_ZH.md](#) (v3.26)

摘要 (Abstract)

erpilot v3.25.10 與 v3.26 完成的 MRP-II 與 CLSP 啟發法雖具有作業研究嚴謹度，但仍是黑盒輸出——客戶無法回答「為什麼下週要拉這麼多 M6 螺絲？」此類根本問題。本文敘述 v3.27 之三重支柱：(i) 資料來源圖 (Provenance Graph) [Cheney, Chiticariu & Tan 2009] — 對任一 planned order 追溯其上游因果鏈；(ii) TOC 五步聚焦法[Goldratt 1984, 1990] — 從 v3.26 之 CRP 載荷自動識別瓶頸並提供處置選項；(iii) OAT 敏感度分析 [Saltelli et al. 2008] — 計算 +20% capacity 之邊際效益。我們論證此三模組形成 IE/Algo/ERP/AI 四域交集之研究級貢獻，並以 12 個結構不變量測試 (含 Schragenheim-Ronen 1990 之 0.85 排隊論閾值) 驗證之。

關鍵字：可解釋 AI、Theory of Constraints、資料來源、敏感度分析、ERP

1. 引言 (Introduction)

1.1 黑盒問題

v3.25.10 + v3.26 之 MRP-II 與 capacity-aware MRP 對應一張數據：

MrpItem (part='M6-BOLT', period='W22')

gross_requirement = 420

net_requirement = 370

planned_order_release = 370

但沒有任何解釋為什麼是 420 而非其他數字。客戶（特別是非 IE 背景的 SMB 老闆）只看到數字，無法判斷：

- 這個 420 是哪張 SO 觸發的？
- 是不是某個 BOM 改了影響到？
- 我們的瓶頸機台目前有多滿？
- 如果加 20% 產能，這個 plan 會變多好？

Vollmann et al. (2005) §11 直言：「使用者不信任他們不理解的計畫」 (*Users will not trust plans they cannot understand*)。

1.2 可解釋 AI 之啟示

ML 領域已有完整的可解釋性研究 [Doshi-Velez & Kim 2017]：

- LIME [Ribeiro, Singh & Guestrin 2016, *KDD*] — 局部線性近似
- SHAP [Lundberg & Lee 2017, *NIPS*] — Shapley value 統一框架
- Counterfactuals [Wachter, Mittelstadt & Russell 2017] — 反事實解釋

這些方法雖針對 ML model 而非 IE/OR 演算法，但其設計哲學（trace causation, decompose contribution, what-if）完全適用於 MRP 黑盒。

1.3 跨域貢獻聲明

本版本之創新點在於將下列四域之概念首次整合於 ERP 規劃情境：

域	借用概念	應用
IE/OR	TOC 五步聚焦 [Goldratt 1984]、Kingman 公式 [Schrageheim & Ronen 1990]	瓶頸自動識別 + 0.85 閾值
演算法	Data provenance [Cheney et al. 2009]、DAG traversal	因果鏈追溯
ERP	MPS-MRP-BOM-Routing 完整資料鏈 [Vollmann 2005]	提供 trace 之資料基礎
可解釋 AI	OAT 敏感度 [Saltelli 2008]、Counterfactuals [Wachter 2017]	反事實 what-if 模擬

2. 方法 (Methodology)

2.1 模組 1：需求來源圖 (Demand Provenance Graph)

問題形式化：給定一個 `MrpItem`（part i, period t, planned_order_release q），找出所有「產生 q 的上游事件之有向圖」。

定義 (forward provenance graph) :

$$G_{\{\text{prov}\}}(i, t) = (V_e, E_e)$$

其中 V_e 為事件節點集合 (SO / MPS / MRP / BOM explosion) , E_e 為「因 → 果」之邊。

節點類型 :

- `mrp_item` : MRP-II 之單期計畫
- `mps_entry` : MPS 之單期主排程
- `bom_explosion` : BOM 一階展開動作
- `sales_order_item` (v3.28 預留)

邊類型 : 每條邊標註轉換係數 :

- `gross_to_net` : $N(t) = G(t) - OH(t) + SS$
- `bom_propagation` : $G_{\{\text{child}\}}(t - L_i) += R_i(t) \cdot q_{\{ic\}} \cdot (1 + s_{\{ic\}})$
- `mps_to_top_demand` : $G_{\{\text{top}\}}(t) += \text{MPS planned_production}(t)$

演算法 : 反向 DFS (demand 上溯)

```
ALGORITHM: explain_planned_order(item)
1. Create root node from `item`
2. For each BOMItem b where b.part_id == item.part_id:
3.   parent_product ← b.product_id
4.   parent_item ← MrpItem in same MRP with part_no == product_no
5.   if parent_item exists:
6.     child_node ← recurse(parent_item, depth - 1)
7.     child_node.label += " 因為要做 ..."
8.     root.children.append(child_node)
9. if no parents found:
10.  mps_node ← look up MPS entries for this part's product equivalent
11.  root.children.append(mps_node)
12. return root
```

複雜度 : $O(D \cdot F)$ where D = max depth, F = avg fan-in 每階。

循環防護 : `max_depth` 參數限制遞迴深度 (預設 5) , 防止 BOM 循環造成 infinite recursion (雖然 BOM 應為 DAG , 但若資料異常仍需 graceful degradation) 。

2.2 模組 2 : TOC 瓶頸分析 (Goldratt's Theory of Constraints)

Goldratt (1984) 提出五步聚焦法 (Five Focusing Steps) :

1. IDENTIFY the system's constraint
2. EXPLOIT the constraint
3. SUBORDINATE everything else to (2)
4. ELEVATE the constraint
5. REPEAT (if (4) succeeded, find new constraint)

閾值之 0.85 由來：

來自 Kingman 公式 [Kingman 1961, *Math Proc Camb Phil Soc* 57] · G/G/1 queue 之平均等待時間：

$$W_q \approx \left(\frac{c_a^2 + c_s^2}{2} \right) \cdot \frac{\rho}{1 - \rho} \cdot \tau$$

當 $\rho \rightarrow 1$ (utilization) · $W_q \rightarrow \infty$ 。實務經驗 (Schragenheim & Ronen 1990) 發現 $\rho > 0.85$ 時 W_q 急遽上升 · 因此將 0.85 設為 **bottleneck-warning threshold**。

erpilot 實作：

```
identify_bottlenecks_from_loads(loads, work_centers):
    group loads by work_center
    for each WC:
        peak_util = max(L.utilization for L in WC's loads)
        is_bottleneck = (peak_util > 0.85)
        if is_bottleneck:
            elevation = [
                "加班 (OT) ",
                "替代機台群 (alternate_group) " if WC has,
                "外包",
                "資本升級",
            ]
            shadow_price = 1.0 if overloaded else 0.0 # LP shadow price
        sort by peak_util desc
```

Shadow Price 之計算：源自 LP duality。在 binding constraint 下 · 每增加 1 單位 RHS (capacity) 使 objective 改善之邊際值即為 shadow price。對 OR 入門級 SMB 客戶 · 我們採極簡近似：超載時 shadow price = 1 (每增 1 min 直接增 1 min 產出) · 非超載時 = 0。

2.3 模組 3：反事實敏感度 (Counterfactual Sensitivity)

問題：給定 work-center k 之 capacity multiplier $\alpha > 1$ (如 1.2 表 +20%) · 計算對 plan 之影響。

方法 (OAT, one-factor-at-a-time)：

1. 跑 baseline plan
2. 修改 $C_{kt} \rightarrow \alpha \cdot C_{kt}$ for the target WC
3. 重跑 Dixon-Silver (其他輸入不變)
4. 比較：overload count / infeasible count / holding cost penalty 之 delta

限制 (Saltelli et al. 2008, §2.5)：

- OAT 不抓 交互作用；同時改變多個 WC 之效應 $\neq$ 個別效應加總
- Global SA 需 Sobol 指數 或 Morris elementary effects (後續 sprint)

為何仍採 OAT：

1. SMB 客戶之直覺問題多為「我加這台機器產能會怎樣」(單變數)
2. Sobol 指數需 $N(d+2)$ 次模擬 (d = factor 數) · 對 ~10 個 WC $\times$ 12 period 規模偏重

3. Saltelli 自己也承認：「OAT in well-understood systems can be informative」

3. 實作 (Implementation)

```
backend/app/services/plan_explanation.py    (~500 行)
├─ ExplanationNode (dataclass + render_tree + to_dict)
├─ explain_planned_order(db, mrp_item_id, max_depth=5)
├─ _explain_via_mps(db, part_id, period, mrp_master_id) [helper]
├─ BottleneckReport (dataclass)
├─ BOTTLENECK_UTILIZATION_THRESHOLD = 0.85 ← Schragenheim-Ronen 1990
├─ identify_bottlenecks_from_loads(loads, work_centers)
├─ identify_bottlenecks(db, mps_id) → (reports, capacity_result)
├─ CounterfactualResult (dataclass)
├─ counterfactual_capacity_increase(db, mps_id, wc_id, multiplier=1.2)
```

整合圖：



4. 驗證 (Validation)

4.1 結構不變量測試 (12 case , 全 pass)

測試	驗證主張
<code>explanation_node_to_dict_roundtrip</code>	序列化正確、遞迴展開
<code>explanation_node_render_tree_shows_hierarchy</code>	ASCII 樹之深度縮排
<code>bottleneck_threshold_at_goldratt_0_85</code>	閾值符合 Schragenheim-Ronen 1990
<code>bottleneck_identification_basic</code>	超載 WC 被正確 flag
<code>bottleneck_elevation_options_only_when_bottleneck</code>	非瓶頸無 elevation 建議
<code>bottleneck_elevation_includes_alternate_group_when_set</code>	alternate_group 出現於 options
<code>bottleneck_shadow_price_positive_at_overload</code>	LP duality : binding $\Rightarrow$ shadow > 0
<code>bottleneck_threshold_boundary</code>	嚴格 > 0.85 (非 $\geq$)
<code>bottleneck_sorted_by_peak_descending</code>	報告依 peak_util desc
<code>explain_planned_order_traces_to_mps</code>	端到端 : MRP $\rightarrow$ tree $\rightarrow$ MPS
<code>explanation_max_depth_clamps</code>	max_depth=0 不遞迴
<code>explain_nonexistent_mrp_item_returns_error_node</code>	找不到時 graceful return

4.2 結果

12/12 tests pass at v3.27 release 。 全 sprint 累計 : 391/391 smoke tests pass 。

5. 限制與未來工作 (Limitations and Future Work)

5.1 本版本不支援

議題	為何不做	將來方向
SO/客戶訂單追溯 (Module 1 第 1 階)	目前由 MPS 起點上溯；未連到客戶 SO	v3.28 : MPS_Entry ↔ SO_Item soft link
多 factor 同時 what-if (OAT 限制)	Global SA 較重	Sobol indices (後續研究)
隨機 what-if	確定性 baseline	Monte Carlo simulation (v3.29 stochastic)
TOC Drum-Buffer-Rope 排程	僅做 identification 未做 scheduling	DBR full impl (v3.28+)
Throughput Accounting	未量化 throughput / OE / I 比例	Throughput Accounting 模組
LLM-generated explanation	目前回傳 ASCII tree + dict ； LLM 包裝待補	tool registry 新增 explain tool
Provenance edge weighting	圖結構平等對待；未量化「主因 vs 次因」	Shapley values for OR (Lundberg-Lee 1.5 extension)

5.2 邊界情況

1. 多階半成品之 BOM：當 part 是另一 Product 的中間品時 (依 v3.25.9 之 part_no == product_no 慣例)，我們會繼續追溯。但若慣例不成立則中斷。
2. MRP master 找不到 MPS：fallback graceful。
3. 空 elevation_options：當非瓶頸時不給建議，避免噪訊。

6. 法律與責任聲明 (Legal Notice / 法律聲明)

⚠ 重要：解釋性輸出之法律性質

本模組產生解釋性與建議性分析，建立於 v3.25.10 + v3.26 之 IE/OR 演算法輸出之上。其結果屬輔助決策資訊，不構成：

1. 不構成事件因果之法律認定：本模組之「provenance graph」識別資料庫於查詢時點之 lineage 關係，不代表：
 - 法律上之因果關係認定（例：法律訴訟中之 causation）
 - 對工程變更責任歸屬之認定
 - 對供應鏈中斷責任之認定
2. 不構成生產決策之建議：TOC bottleneck identification 採 Goldratt 啟發式框架，其輸出為統計性訊號，不代表：
 - 必須立刻採取加班 / 外包之指示
 - 對 operator 工時安排之要求
 - 對機台採購之資本支出建議（capital expenditure 決策應有專責財務評估）
3. 不構成市場行為之保證：counterfactual sensitivity 採 OAT 局部敏感度，假設其他變數不變。實務上：
 - 加 capacity 後可能影響供應商價格、operator 招募
 - 多變數交互作用未模型化（見 Saltelli 2008 §2.5）
 - 結果僅供「直覺方向」參考，不可外推為精確預測
4. LLM 包裝層警告（若後續啟用）：當 explanation 經 LLM 改寫為自然語言時：
 - LLM 可能 hallucinate（幻覺）：產出與 raw data 不符之描述
 - LLM 可能簡化過度：丟失重要 nuance
 - 凡 LLM 改寫之內容，建議客戶以本模組之 raw data（to_dict() / render_tree()）為準
5. 演算法限制：
 - Provenance：依資料庫快照，未處理時間旅行查詢（time-travel queries）
 - TOC：0.85 閾值源自 G/G/1 queue 假設；多 server / batch / priority 情境可能不同
 - OAT：不抓 factor 間交互作用 [Saltelli 2008]
6. 不擔保條款：於適用法律所允許之最大範圍內（to the maximum extent permitted by applicable law），erpilot 對下列事項不承擔責任：
 - 因採用本模組解釋而對 supplier / customer / employee 之錯誤指控
 - 因 TOC bottleneck mis-identification 所造成之 capacity 投資失誤
 - 因 counterfactual 與實際偏離所造成之規劃失準
 - LLM 改寫層之 hallucination 後果
 - 第三方依本解釋採取行動所衍生之勞動 / 契約爭議
7. 累積適用前置文件之聲明：本版本疊加於 v3.25.10 + v3.26，因此 [MRP_ALGORITHM_DESIGN_ZH.md](#) §6 及 [MRP_CAPACITY_AWARE_DESIGN_ZH.md](#) §6 之所有聲明於此累積適用。

建議實務做法

- 將 provenance tree 視為「查資料的快捷工具」而非「責任認定報告」
- TOC 建議由生管主管 / 廠長覆核後再決定加班 / 外包 / 升級
- Counterfactual 結果視為「直覺方向參考」而非「精確投資 ROI 報告」
- LLM 改寫之自然語言解釋應與 raw structured data 並列呈現，讓使用者可交叉驗證

7. 文獻 (References)

- [1] Goldratt, E. M. (1984). *The Goal: A Process of Ongoing Improvement*. North River Press. — TOC 創始作
- [2] Goldratt, E. M. (1990). *What Is This Thing Called Theory of Constraints*. North River Press. — TOC 方法論
- [3] Cheney, J., Chiticariu, L., & Tan, W.-C. (2009). Provenance in databases: Why, how, and where. *Foundations and Trends in Databases*, 1(4), 379-474. — Provenance 經典綜述
- [4] Doshi-Velez, F., & Kim, B. (2017). Towards a rigorous science of interpretable machine learning. *arXiv:1702.08608*. — 可解釋 ML 框架
- [5] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *NIPS 30*. — SHAP
- [6] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why Should I Trust You?": Explaining the predictions of any classifier. *KDD '16*. — LIME
- [7] Wachter, S., Mittelstadt, B., & Russell, C. (2017). Counterfactual explanations without opening the black box. *Harv. J. L. & Tech.*, 31, 841. — Counterfactual explanation
- [8] Saltelli, A., Ratto, M., Andres, T., Campolongo, F., Cariboni, J., Gatelli, D., Saisana, M., & Tarantola, S. (2008). *Global Sensitivity Analysis: The Primer*. Wiley. — 敏感度分析
- [9] Kingman, J. F. C. (1961). The single server queue in heavy traffic. *Math. Proc. Cambridge Phil. Soc.*, 57(4), 902-904. — Kingman 公式
- [10] Schragenheim, E., & Ronen, B. (1990). Drum-buffer-rope shop floor control. *Production and Inventory Management Journal*, 31(3), 18-22. — TOC 0.85 閾值之實務經驗
- [11] Mabin, V. J., & Balderstone, S. J. (2003). The performance of the theory of constraints methodology: Analysis and discussion of successful TOC applications. *IJOPM*, 23(6), 568-595.
- [12] Caruana, R., Lou, Y., Gehrke, J., Koch, P., Sturm, M., & Elhadad, N. (2015). Intelligible models for healthcare: Predicting pneumonia risk and hospital 30-day readmission. *KDD '15*. — Intelligibility in critical domains

[13] Vollmann, T. E., Berry, W. L., Whybark, D. C., & Jacobs, F. R. (2005). *Manufacturing Planning and Control for Supply Chain Management* (5th ed.). McGraw-Hill. — §11 「使用者不信任不理解的計畫」

[14] Pinedo, M. L. (2016). *Scheduling: Theory, Algorithms, and Systems* (5th ed.). Springer.

8. 變更紀錄 (Changelog)

版本	日期	變更
v3.25.10	2026-05-20	無產能 MRP-II
v3.26	2026-05-20	產能感知 MRP (Dixon-Silver)
v3.27	2026-05-20	本版本：可解釋規劃 + TOC 瓶頸 + OAT 反事實
將來 v3.28+	TBD	SO 追溯 + LLM 包裝層 + Drum-Buffer-Rope
將來 v3.29+	TBD	Sobol 全域敏感度 + Monte Carlo

最後更新：2026-05-20 (v3.27) 作者：erpilot 工程團隊 (含 IE/OR/AI 跨域學術方法論引用) 版本：1.0

English version： [PLANNING_EXPLAINABILITY_DESIGN_EN.md](#)